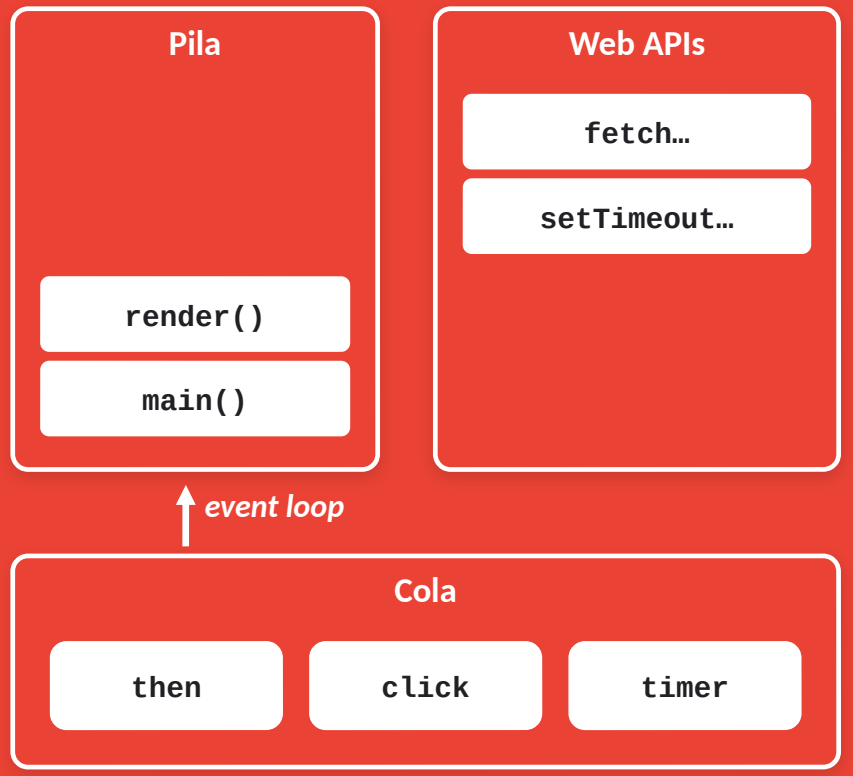
MÓDULO DWEC · DAW — UNIDAD DIDÁCTICA 6

Asincronía y almacenamiento

RA4 · Event loop, promesas, async/await, fetch, localStorage y Vite

- Entender por qué JavaScript no se bloquea esperando a la red
- Pedir y enviar datos a una API con fetch, async/await y errores bien tratados
- Recordar los datos en el navegador y preparar el proyecto con Vite



Una sola pila, varias colas y un bucle que las va vaciando.

DÓNDE ESTAMOS

La aplicación ya se ve; ahora tiene que recordar y hablar con el servidor



La pregunta de la unidad: ¿cómo pido datos a un servidor sin congelar la página, y cómo hago que mi aplicación se acuerde de todo al recargar?

OBJETIVO — Calentamiento C de la Práctica 2 (vie 20/11): el tablero guarda sus tareas en localStorage e importa tareas con fetch. Parcial 2: mar 24/11.

CONTEXTO PROFESIONAL

Lo que pasa cuando abres una app web

Al abrir Trello, Spotify o el campus virtual

`0 ms`	La página aparece con un esqueleto vacío
`50 ms`	Lee tus preferencias guardadas (localStorage)
`60 ms`	Pide tus datos a la API (fetch)… y NO se congela
`400 ms`	Llega la respuesta JSON y se pinta (UD5)
`—`	Si falla la red: mensaje y botón «reintentar»

Todo es asíncrono

Red, temporizadores, archivos, permisos: nada responde al instante

Todo habla con APIs

El frontend pide y envía JSON por HTTP; el backend puede estar en cualquier lenguaje

Recordar sin servidor

Tema, borradores, carrito o el modo sin conexión viven en el navegador

La red falla

Lenta, caída, 404, 500: una app profesional lo muestra y se recupera

Herramientas de build

Vite prepara el código para producción: un dist/ ligero y con caché

DATO CLAVE — Un buen frontend se nota sobre todo cuando las cosas van mal: con la red lenta, sin conexión o cuando el servidor responde con un error.

HOJA DE RUTA

Seis bloques, del event loop a Vite

1. **Modelo de ejecución** — un hilo, event loop
2. **Promesas** — then, catch, all
3. **async/await** — try/catch, paralelo
4. **fetch y APIs** — HTTP, JSON, errores
5. **Almacenamiento** — localStorage, JSON
6. **Módulos y Vite** — build, .env, import()

Laboratorio

Cartelera con fetch, estados y favoritos guardados

DevTools

Network, throttling, Application

Mini-reto

Pomodoro que sobrevive al recargar

CLAVE — Esta unidad se ve en la pestaña Network de DevTools: cada petición, su código, lo que tarda y lo que devuelve.



BLOQUE 1

El modelo de ejecución

Un solo hilo, la pila, las Web APIs, las colas y el event loop

JavaScript hace una cosa a la vez

Síncrono: esperar bloquea



Ni clics, ni scroll, ni animaciones durante la espera

Asíncrono: pedir y seguir



El navegador espera por ti; JavaScript sigue atendiendo

bloqueo.js

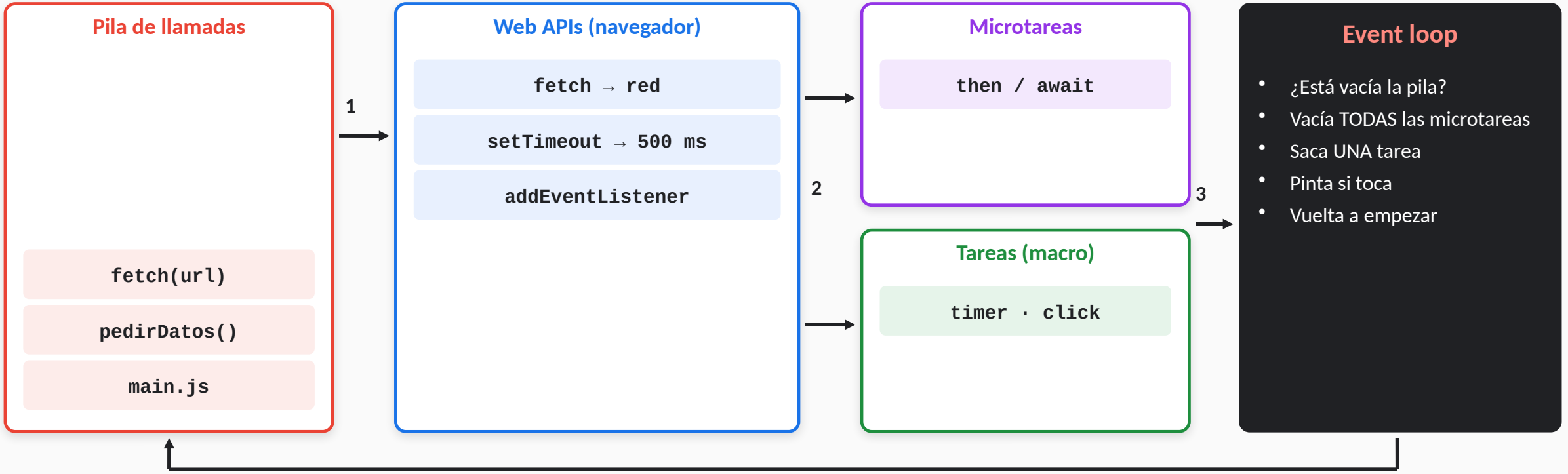
```
// ✗ un bucle de 3 s congela la página entera
const fin = Date.now() + 3000;
while (Date.now() < fin) { /* nada */ }
console.log("ya");

// ✓ «avísame dentro de 3 s» y seguir
setTimeout(() => console.log("ya"), 3000);
```

Un hilo = una pila de llamadas. Mientras tu código está ejecutándose, el navegador no puede pintar ni atender un clic. Por eso lo lento (red, temporizadores, archivos) NO se espera: se pide, y se recibe la respuesta más tarde.

CLAVE — Asíncrono no significa «en paralelo»: tu código sigue siendo de un solo hilo. Lo que va aparte es la espera, que la hace el navegador.

Pila, Web APIs, colas y el bucle que las conecta



1 tu código pide algo al navegador · 2 cuando termina, deja el callback en una cola · 3 el bucle espera a que la pila esté vacía · 4 lo ejecuta

NOTA — Las Web APIs (fetch, temporizadores, eventos del DOM) son del navegador, no del lenguaje. En Node existen otras equivalentes.

setTimeout, setInterval y cómo pararlos

timers.js

```js
const id = setTimeout(() => {
  console.log("una vez, dentro de 1 s");
}, 1000);
clearTimeout(id);          // cancelado

const reloj = setInterval(() => {
  pintarHora(new Date());
}, 1000);
clearInterval(reloj);      // parar
```

orden.js

```js
console.log("A");
setTimeout(() => console.log("B"), 0);
console.log("C");

// A
// C
// B   ← 0 ms no es «ya»: es «cuando
//       la pila esté vacía»
```

	Qué devuelve	Ojo
`setTimeout(f, ms)`	Un id para cancelarlo	ms es el MÍNIMO, no exacto
`setInterval(f, ms)`	Un id	Si no lo paras, sigue para siempre
`clearTimeout / clearInterval`	—	Con el id que devolvieron

CLAVE — Un temporizador no es un reloj fiable: si la pila está ocupada, espera. Para medir tiempo de verdad, compara con Date.now() (mini-reto).

Las promesas se cuelan antes que los temporizadores

consola

```
console.log("1 · síncrono");

setTimeout(() => console.log("2 · tarea"), 0);

Promise.resolve().then(() =>
  console.log("3 · microtarea"));

console.log("4 · síncrono");
```

Orden en que sale:

1. `1 · síncrono`
2. `4 · síncrono`
3. `3 · microtarea`
4. `2 · tarea`

Para practicar: cambiad el orden de las líneas y predecid la salida antes de ejecutar.

La regla: primero todo el código síncrono; luego TODAS las microtareas (then, await); y solo entonces la siguiente tarea (temporizador, clic).

NOTA — Por eso un bucle infinito de microtareas también congela la página: el bucle nunca llega a pintar.

Así se hacía antes de las promesas

piramide.js

```
obtenerUsuario(id, (err, usuario) => {
  if (err) return mostrarError(err);
  obtenerTablero(usuario.tablero, (err, t) => {
    if (err) return mostrarError(err);
    obtenerTareas(t.id, (err, tareas) => {
      if (err) return mostrarError(err);
      guardar(tareas, (err) => {
        if (err) return mostrarError(err);
        render(tareas);
      });
    });
  });
});
```

Pirámide

cada paso dentro del anterior

Errores repetidos

un if (err) en cada nivel

Difícil de combinar

¿y si quiero 3 a la vez?

con async/await (bloque 3)

```
const u = await obtenerUsuario(id);
const t = await obtenerTablero(u.tablero); …
```

CLAVE — Los callbacks siguen existiendo (addEventListener, setTimeout). Para encadenar operaciones asíncronas, hoy se usan promesas y async/await.

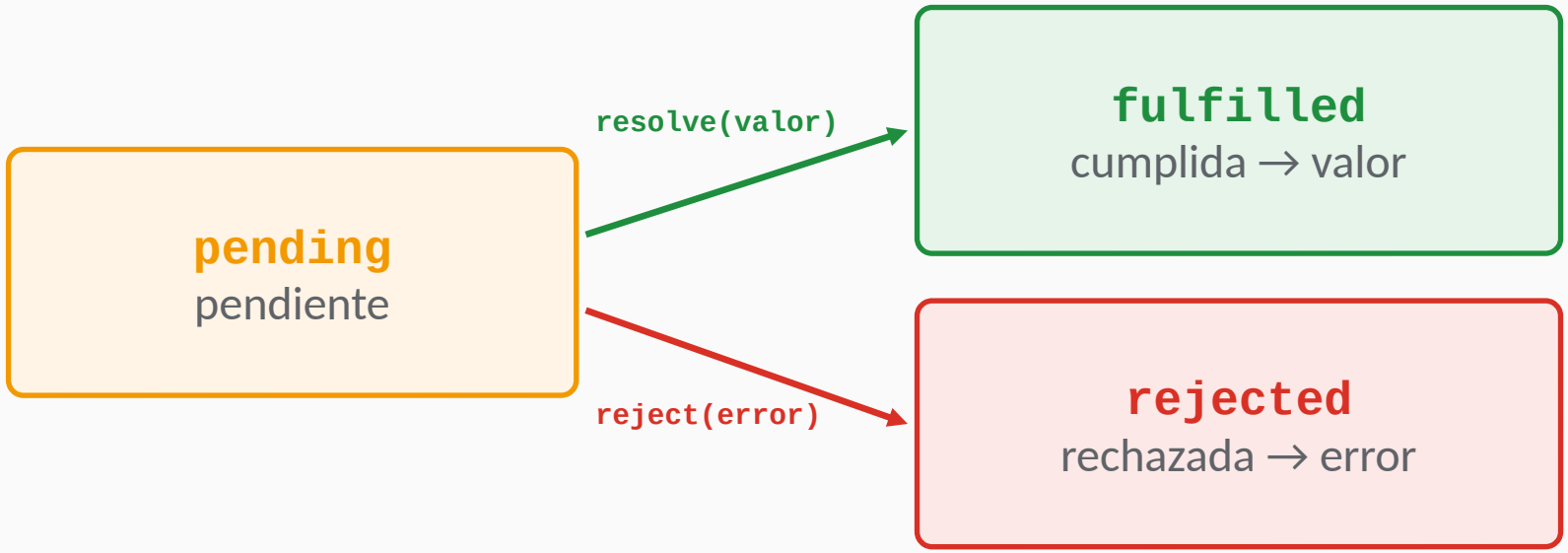


BLOQUE 2

Promesas

Un valor que llegará: estados, crearlas, consumirlas, encadenarlas y combinarlas

Un objeto que representa un resultado que todavía no está




```
consola
const p = fetch("/datos.json");
p;
// Promise {<pending>}

// … un momento después
p;
// Promise {<fulfilled>:
//   Response}
```

La metáfora: el tique de la cafetería. No tienes el café, pero tienes algo que te avisará cuando esté (o de que se han quedado sin leche). Mientras, puedes hacer otras cosas.

CLAVE — Una promesa se cumple o se rechaza UNA sola vez. Si ya se resolvió, un then nuevo recibe el valor al momento (como microtarea).

BLOQUE 2 · CREAR UNA PROMESA

new Promise((resolve, reject) => { … })

esperar.js

```
// convertir setTimeout en una promesa
function esperar(ms) {
  return new Promise((resolve) => {
    setTimeout(resolve, ms);
  });
}

function tirarDado() {
  return new Promise((resolve, reject) => {
    const n = Math.ceil(Math.random() * 6);
    n === 1 ? reject(new Error("¡Un 1!")) : resolve(n);
  });
}
```

Lo que hay que saber

- La función de dentro (ejecutor) se ejecuta YA, de forma síncrona
- resolve(valor): cumplida. reject(error): rechazada
- Rechaza siempre con new Error(…), como en la UD4
- Llamar a resolve dos veces no hace nada: solo cuenta la primera
- Casi nunca escribiréis new Promise: fetch y las APIs modernas ya devuelven promesas

```
esperar(500).then(() => console.log("medio segundo después"));
```

NOTA — esperar(ms) es muy útil para simular una red lenta en el laboratorio: await esperar(800) antes de devolver los datos.

Decir qué hacer cuando la promesa se resuelva

consumir.js

```
mostrarCargando(true);
tirarDado()
  .then((n) => console.log("Has sacado", n))
  .catch((error) => console.warn(error.message))
  .finally(() => mostrarCargando(false));

console.log("Esto sale ANTES que el dado");
```

`then(f)` si se cumple: f recibe el valor

`catch(f)` si se rechaza: f recibe el error

`finally(f)` siempre, al terminar (sin valor)

Cada then devuelve una promesa NUEVA. Por eso se pueden encadenar con puntos, y por eso un solo catch al final recoge el error de cualquier paso anterior (siguiente diapositiva).

CLAVE — El console.log de la última línea sale antes porque then solo REGISTRA qué hacer: el código sigue y el then se ejecuta después, como microtarea.

Lo que devuelves en un then es lo que recibe el siguiente

cadena.js

```
fetch("/api/usuario")
  .then((r) => r.json())          // devuelve una promesa
  .then((u) => fetch(`/api/tableros/${u.tablero}`))
  .then((r) => r.json())
  .then((tablero) => render(tablero))
  .catch((e) => mostrarError(e));  // el de CUALQUIER paso
```

olvido.js

```
// ✗ sin return: el siguiente recibe undefined
fetch(url)
  .then((r) => { r.json(); })
  .then((datos) => console.log(datos)); // undefined

// ✓ con llaves, return; sin llaves, implícito
  .then((r) => { return r.json(); })
  .then((r) => r.json())
```

Si devuelves valor normal: pasa tal cual al siguiente then

Si devuelves una promesa: se espera y pasa SU valor (se «aplana»)

Si devuelves throw / rechazo: salta todos los then hasta el catch

CLAVE — El error n.º 1 con promesas: un then con llaves y sin return. Todo «funciona», pero el siguiente paso recibe undefined.

Promise.all, allSettled, race y any

Método	Se cumple cuando...	Se rechaza cuando...	Úsalo para
`Promise.all([…])`	TODAS se cumplen → array de valores (en orden)	La PRIMERA falla	Cargar varias cosas que hacen falta todas
`Promise.allSettled([…])`	Todas terminan, bien o mal → [{status, value|reason}]	Nunca	Informe de qué ha ido bien y qué no
`Promise.race([…])`	La primera que TERMINE (bien o mal)	Si la primera en terminar falla	Tiempo límite
`Promise.any([…])`	La primera que SE CUMPLA	Todas fallan (AggregateError)	Varios servidores espejo

all.js

```
const [usuario, tareas] = await Promise.all([
  getJSON("/api/usuario"), getJSON("/api/tareas"),
]);
```

Todas empiezan a la vez: Promise.all no las lanza en orden; solo espera. Tarda lo que la más lenta.

NOTA — El orden del array de resultados es el de las promesas que pasaste, no el orden en que terminaron.

Un throw dentro de un then rechaza la cadena

errores.js

```
Promise.resolve(5)
  .then((n) => { throw new Error("falla en " + n); })
  .then(() => console.log("no se ejecuta"))
  .catch((e) => { console.warn(e.message); return 0; })
  .then((n) => console.log("sigue con", n));
// falla en 5
// sigue con 0
```

Tres reglas

- throw dentro de then = rechazo: salta al siguiente catch
- catch RECUPERA: lo que devuelve sigue la cadena como cumplida
- Si quieres que siga fallando, vuelve a lanzar: throw e

Uncaught (in promise): un rechazo sin catch. No para la página, pero el error se pierde y el usuario se queda mirando un «Cargando…» eterno. Toda cadena termina en catch (o en try/catch con await).

CLAVE — Un try/catch normal NO captura el error de una promesa que no esperas: try { fetch(url) } catch… no ve nada. Hace falta .catch o await.



BLOQUE 3

async / await

Escribir código asíncrono como si fuera síncrono: try/catch, serie frente a paralelo y trampas

Promesas, pero leídas de arriba abajo

con then

```
function cargar() {
  return fetch("/datos/tareas.json")
    .then((r) => r.json())
    .then((tareas) => {
      render(tareas);
      return tareas.length;
    });
}
```

con async/await

```
async function cargar() {
  const r = await fetch("/datos/tareas.json");
  const tareas = await r.json();
  render(tareas);
  return tareas.length;
}
```

async	la función devuelve SIEMPRE una promesa (su return es el valor)
await	pausa ESTA función hasta que la promesa se resuelva y da su valor
¿dónde?	dentro de funciones async, o en el nivel superior de un módulo

CLAVE — await no congela la página: pausa solo esa función. Mientras espera, el event loop sigue atendiendo clics y pintando.

try / catch / finally, como el código de toda la vida

cargar.js

```
async function cargarTareas() {
  estado.textContent = "Cargando…";
  boton.disabled = true;
  try {
    const tareas = await getJSON("/datos/tareas.json");
    render(tareas);
    estado.textContent = "";
  } catch (error) {
    estado.textContent = `Error: ${error.message}`;
  } finally {
    boton.disabled = false;   // pase lo que pase
  }
}
```

Dónde poner el try

- Alrededor de lo que puede fallar y donde SABES qué hacer con el error
- En el nivel de la interfaz: ahí puedes mostrar el mensaje
- Las funciones de bajo nivel (getJSON) lanzan; la interfaz decide
- finally para lo que hay que deshacer siempre: el botón, el spinner

NOTA — Una función async que lanza devuelve una promesa rechazada: quien la llame con await la captura con su propio try/catch.

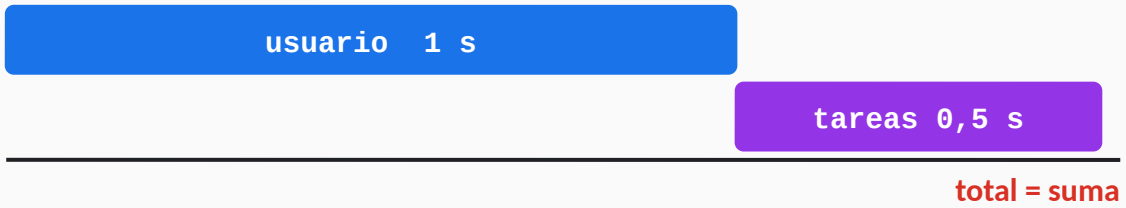
Dos await seguidos esperan uno detrás de otro

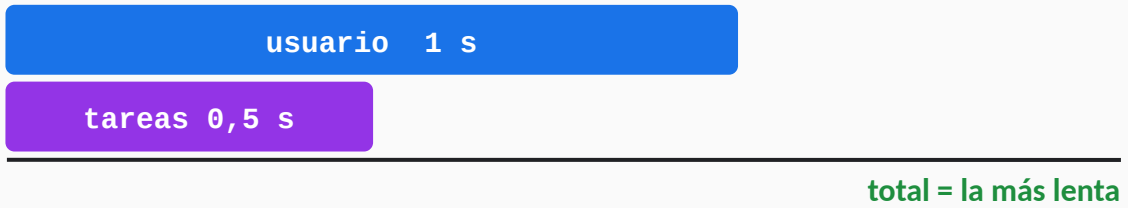
serie.js ≈ 1,5 s

```
const u = await getJSON("/api/usuario");  // 1 s
const t = await getJSON("/api/tareas");   // 0,5 s
```

paralelo.js ≈ 1 s

```
const [u, t] = await Promise.all([
  getJSON("/api/usuario"), getJSON("/api/tareas"),
]);
```





¿Cuándo en serie? solo cuando la segunda petición NECESITA el resultado de la primera (el id del tablero del usuario). Si son independientes, Promise.all.

CLAVE — await en un bucle for…of también va en serie: bien si hay que respetar el orden, lento si no. Para muchas a la vez: Promise.all(lista.map(…)).

Las tres que vais a cometer

1 · Olvidar await

const datos = getJSON(url);
datos.length → undefined
datos → Promise {<pending>}

2 · await dentro de forEach

forEach NO espera a funciones async: termina enseguida y las peticiones van sueltas, sin orden y sin capturar sus errores.

3 · async sin catch

Si nadie hace await ni .catch, el rechazo sale como «Uncaught (in promise)» y la interfaz se queda colgada.

bien.js

```
// ✓ en orden, de una en una
for (const t of tareas) await guardar(t);

// ✓ todas a la vez
await Promise.all(tareas.map((t) => guardar(t)));
```

main.js (módulo)

```
// await en el nivel superior: solo en módulos
const config = await getJSON("/config.json");
iniciar(config);
// el resto de módulos que lo importen esperan
```

NOTA — En la consola de Chrome se puede escribir await directamente, sin async. En vuestro código, solo en funciones async o en el nivel superior de un módulo.



BLOQUE 4

fetch y APIs

HTTP en una diapositiva, JSON, GET y POST, el 404 que no falla, estados de la interfaz, cancelar y CORS

Petición y respuesta

PETICIÓN (el navegador)

```
POST /api/tareas HTTP/1.1
Content-Type: application/json
Authorization: Bearer …

{"titulo":"Entregar","prioridad":"alta"}
```

RESPUESTA (el servidor)

```
HTTP/1.1 201 Created
Content-Type: application/json

{"id":42,"titulo":"Entregar",…}
```

Método	Para
`GET`	leer
`POST`	crear
`PUT` · `PATCH`	reemplazar · cambiar
`DELETE`	borrar

Código	Significa
`200` · `201` · `204`	OK · creado · OK sin cuerpo
`400` · `401` · `403`	datos mal · sin sesión · sin permiso
`404`	no existe
`500` · `503`	fallo del servidor · no disponible

CLAVE — 2xx = bien · 4xx = la culpa es de la petición (tuya) · 5xx = la culpa es del servidor. Lo veréis todo en la pestaña Network.

El formato en el que viajan los datos

json.js

```
const texto = JSON.stringify(
  { id: 1, titulo: "A", hecha: false });
// '{"id":1,"titulo":"A","hecha":false}'

const obj = JSON.parse(texto);   // de vuelta a objeto
JSON.parse("{titulo: 'A'}");    // SyntaxError
```

Valor en JS	En JSON	Al volver con parse
string, number, boolean, null	igual	igual
array, objeto literal	igual	igual
Date	"2026-11-20T08:00:00.000Z"	¡un string!
undefined, funciones	desaparecen	—
instancia de una clase	solo sus propiedades públicas	objeto normal: sin métodos ni #privados
NaN, Infinity	null	null

Reglas del formato: claves SIEMPRE entre comillas dobles, sin comillas simples, sin comas finales, sin comentarios. Es texto, no JavaScript: por eso hay que convertirlo con parse y stringify.

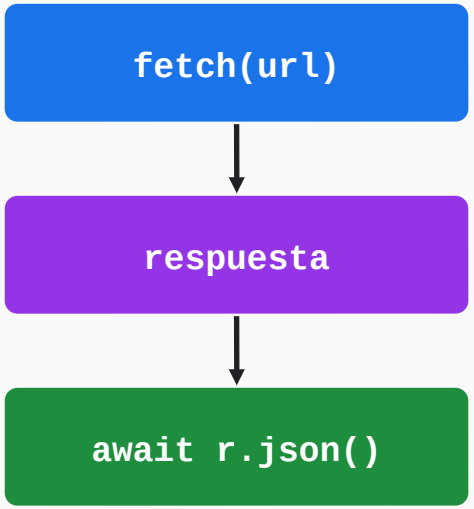
CLAVE — Las fechas y las clases no sobreviven a JSON: hay que reconstruirlas al leer (new Date(texto), Tablero.desde(datos)). Lo usaréis en el bloque 5.

Dos await: uno para la respuesta, otro para el cuerpo

get.js

```
const respuesta = await fetch("/datos/tareas.json");
respuesta.ok;        // true si el código es 200–299
respuesta.status;    // 200
respuesta.headers.get("Content-Type");

const tareas = await respuesta.json(); // cuerpo parseado
```



La promesa se cumple al llegar las CABECERAS

Objeto Response: ok, status, headers…

Lee el CUERPO entero y lo parsea

Para leer el cuerpo como…	Método
JSON (lo normal)	await r.json()
texto	await r.text()
archivo / imagen	await r.blob()

El cuerpo solo se lee UNA vez: llamar a r.json() dos veces lanza TypeError (body stream already read).

NOTA — Los ficheros de la carpeta public/ de Vite se sirven en la raíz: public/datos/tareas.json → fetch("/datos/tareas.json").

BLOQUE 4 · FETCH NO FALLA CON UN 404

Solo rechaza si NO hay respuesta (sin red, CORS, DNS)

Qué pasa	¿fetch rechaza?	respuesta.ok
200 OK	No	true
404 No existe	No	false
500 Error del servidor	No	false
Sin conexión / servidor caído	Sí: TypeError	—
Bloqueado por CORS	Sí: TypeError	—

Sin el if (!r.ok): un 404 llega a r.json() con una página HTML de error y revienta con un SyntaxError que no dice nada útil.

api.js

```
export async function getJSON(url) {
  const r = await fetch(url);
  if (!r.ok) {
    throw new Error(`HTTP ${r.status} en ${url}`);
  }
  const tipo = r.headers.get("Content-Type") ?? "";
  if (!tipo.includes("json")) {
    throw new Error(`${url} no es JSON`);
  }
  return r.json();
}
```

Ojo, Vite en desarrollo: un fichero que no existe responde 200 con el index.html. Por eso se mira el Content-Type.

CLAVE — fetch rechaza solo por fallos de red. Los errores HTTP (4xx, 5xx) son respuestas «normales»: los tienes que convertir tú en error.

POST, PUT, PATCH y DELETE con fetch

post.js

```
export async function postJSON(url, datos) {
  const r = await fetch(url, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(datos),
  });
  if (!r.ok) throw new Error(`HTTP ${r.status}`);
  return r.json();   // 201: el recurso creado, con su id
}

await fetch(`/api/tareas/${id}`, { method: "DELETE" });
```

Las tres piezas

- method: por defecto es GET
- headers: Content-Type dice que el cuerpo es JSON
- body: un STRING, por eso JSON.stringify (un objeto tal cual no vale)
- La respuesta suele traer el objeto guardado con su id definitivo

Para practicar sin servidor propio: jsonplaceholder.typicode.com acepta POST y responde 201 (no guarda nada de verdad).

NOTA — Con FormData (UD5) también se puede enviar: body: new FormData(form), y entonces NO pongas Content-Type: lo pone el navegador.

Cargando, error, vacío y datos: los cuatro, siempre

Cargando

«Cargando…» o un esqueleto · botón desactivado · aria-busy

Error

mensaje que se entiende + botón «Reintentar»

Vacío

«Aún no hay tareas» + qué hacer para crear la primera

Datos

la lista pintada (UD5)

estado.js

```
function pintarEstado({ cargando, error, tareas }) {
  zona.setAttribute("aria-busy", String(cargando));
  if (cargando) return mostrar("Cargando…");
  if (error) return mostrar(error, { reintentar: true });
  if (tareas.length === 0) return mostrar("Aún no hay tareas");
  render(tareas);
}
```

Probadlo: Network → Slow 4G para ver el «Cargando…» y Offline para el error.

CLAVE — Si solo programas el caso feliz (datos), la app parece rota en cuanto la red es lenta: el usuario no sabe si está cargando o si ha fallado.

AbortController y AbortSignal.timeout

buscar.js

```
let controlador;
buscador.addEventListener("input", async () => {
  controlador?.abort();            // cancela la anterior
  controlador = new AbortController();
  try {
    const url = `/api/buscar?q=${buscador.value}`;
    const r = await fetch(url,
      { signal: controlador.signal });
    render(await r.json());
  } catch (e) {
    if (e.name !== "AbortError") mostrarError(e);
  }
});
```

timeout.js

```
// rechaza si tarda más de 5 s
await fetch(url, { signal: AbortSignal.timeout(5000)
});
// e.name === "TimeoutError"
```

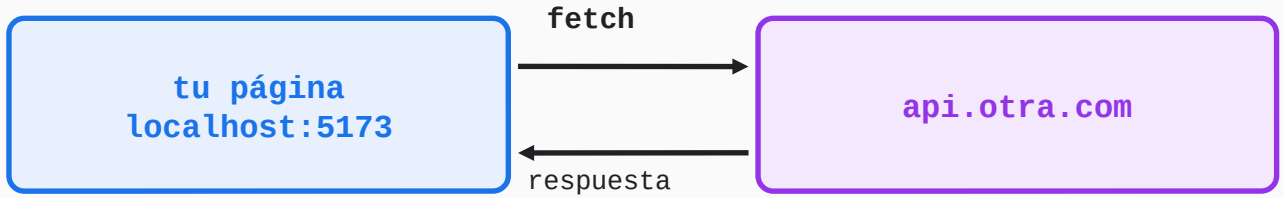
Por qué cancelar: si el usuario escribe «ra», «rat», «rató», la respuesta de «ra» puede llegar la ÚLTIMA y pisar a la buena.

Mejor aún, no pedir en cada tecla: espera 300 ms sin escribir (debounce con setTimeout y clearTimeout) y entonces pide. Menos peticiones y menos carreras.

NOTA — Abortar rechaza la promesa con un error de nombre AbortError: no es un fallo, así que no se lo muestres al usuario.

BLOQUE 4 · CORS

El navegador no deja leer respuestas de otro origen… salvo que el servidor lo permita




```
consola
Access to fetch at 'https://api.otra.com/…'
from origin 'http://localhost:5173' has been
blocked by CORS policy: No 'Access-Control-
Allow-Origin' header is present…
```

Si responde con `Access-Control-Allow-Origin: http://localhost:5173`
→ el navegador te deja leerla ✓

Si no la envía: la petición llegó y el servidor respondió, pero el navegador NO deja que tu código lea la respuesta → fetch rechaza con TypeError.

Lo que hay que saber

- Origen = protocolo + dominio + puerto: localhost:5173 y localhost:3000 son distintos
- Lo decide el SERVIDOR con la cabecera Access-Control-Allow-Origin
- No se arregla desde el frontend (ni con un plugin del navegador en producción)
- En desarrollo: server.proxy de Vite

CLAVE — CORS no es un error de vuestro código: es una protección del navegador. Si una API pública no envía la cabecera, no se puede usar desde el navegador.



BLOQUE 5

Almacenamiento en el navegador

localStorage y sessionStorage, guardar objetos y clases, límites, seguridad y el evento storage

Cuatro sitios, cuatro usos

	localStorage	sessionStorage	Cookies	IndexedDB
Cuánto dura	Hasta que se borre	Hasta cerrar la pestaña	Hasta su fecha de caducidad	Hasta que se borre
Capacidad	≈ 5 MB por origen	≈ 5 MB por origen	≈ 4 KB cada una	Cientos de MB
¿Viaja al servidor?	No	No	Sí, en CADA petición	No
API	Síncrona, solo strings	Síncrona, solo strings	document.cookie (incómoda)	Asíncrona, objetos
Para	Preferencias, borradores, el tablero	Un asistente en varios pasos	Sesión (las pone el servidor)	Datos grandes, modo offline

CLAVE — En esta unidad, localStorage: es lo más sencillo y basta para el tablero. IndexedDB y las cookies, solo para saber que existen y cuándo se usan.

Cinco métodos y una regla: todo son strings

storage.js

```
localStorage.setItem("tema", "oscuro");
localStorage.getItem("tema");     // "oscuro"
localStorage.getItem("no-existe"); // null
localStorage.removeItem("tema");
localStorage.clear();             // TODO lo del origen
localStorage.length;              // cuántas claves

localStorage.setItem("visitas", 3);
localStorage.getItem("visitas");  // "3"  ← string
```

Reglas

- Todo se guarda como string: 3 se guarda como "3"
- getItem de algo que no existe devuelve null, no undefined ni error
- Es POR ORIGEN: localhost:5173 (dev) y localhost:4173 (preview) tienen datos distintos
- sessionStorage tiene la misma API

Prefijad las claves con el nombre de la app ("tablero:tareas"): en localhost conviven todos vuestros proyectos de clase y se pisarían.

NOTA — En DevTools → Application → Local Storage veis, editáis y borráis las claves a mano. Muy útil para probar qué pasa con datos raros.

stringify al guardar, parse al leer, y desconfiar de lo leído

almacen.js

```js
const CLAVE = "tablero:preferencias";
const DEFECTO = { tema: "claro" };

export function guardar(prefs) {
  localStorage.setItem(CLAVE, JSON.stringify(prefs));
}

export function cargar() {
  try {
    const texto = localStorage.getItem(CLAVE);
    return JSON.parse(texto) ?? DEFECTO;
  } catch {
    return DEFECTO;   // texto corrupto
  }
}
```

Sin stringify

setItem("x", { a: 1 }) guarda "[object Object]"

parse(null)

devuelve null: por eso el ?? con el valor por defecto

Texto corrupto

lo editó el usuario o una versión vieja: parse lanza SyntaxError

CLAVE — Lo que viene de localStorage es como lo que escribe el usuario (UD5): puede ser cualquier cosa. Valida al cargar.

JSON pierde los métodos y los #privados: hay que reconstruir

el problema

```
const t = new Tarea("A", "alta");
JSON.stringify(t);
// '{"id":1,"limite":null,"creada":"2026-…"}'
//  ← ¡sin titulo ni prioridad! (son #privados)
const copia = JSON.parse("…");
copia.subirPrioridad();  // TypeError: no es una Tarea
```

El patrón: toJSON() decide qué se guarda (JSON.stringify lo llama solo) y static desde(datos) reconstruye la instancia, validando otra vez en el constructor. Lo vimos con Tablero.desde en la UD4.

Tarea.js

```
class Tarea {
  // …
  toJSON() {          // lo que se guarda
    return {
      titulo: this.titulo,
      prioridad: this.prioridad,
      limite: this.limite
        ? fechaISO(this.limite) : null,
    };
  }
  static desde(datos) {   // reconstruir
    return new Tarea(datos.titulo,
      datos.prioridad, datos.limite);
  }
}
```

CLAVE — Bloque C de la Práctica 2: el tablero se guarda con toJSON y se reconstruye con Tablero.desdeJSON al cargar la página.

Lo que localStorage no es

≈ 5 MB por origen

Si se llena, setItem lanza QuotaExceededError: try/catch al guardar

Es síncrono

Guardar 4 MB en cada tecla bloquea la página: guarda al cambiar algo, no en un bucle

No es privado

Cualquier script de la página lo lee. Con un XSS (UD5), el atacante se lo lleva todo

Nunca tokens ni contraseñas

La sesión, en cookies HttpOnly que pone el servidor y JavaScript no puede leer

No se sincroniza

Es de ese navegador en ese ordenador: si cambias de móvil, no está

Se puede borrar

El usuario, el modo incógnito o el navegador por falta de espacio

CLAVE — localStorage es una libreta en el navegador del usuario: cómoda para recordar cosas suyas, inútil para guardar secretos o datos que deban estar seguros.

Sincronizar varias pestañas

Pestaña A

`setItem("tablero:tareas", …)`

A NO recibe su propio evento

→

Pestaña B

recibe «storage»
→ recarga y repinta

sincronizar.js

```
addEventListener("storage", (e) => {
  if (e.key !== "tablero:tareas")
    return;
  tablero = cargarTablero();
  render();
});
```

e.key, e.oldValue y e.newValue dicen qué cambió. Solo salta en las OTRAS pestañas del mismo origen, nunca en la que hizo el cambio. Así, si tienes el tablero abierto dos veces, las dos ven lo mismo.

NOTA — Es opcional para la práctica, pero muy vistoso en clase: dos ventanas lado a lado, mueve una tarea en una y aparece movida en la otra.



BLOQUE 6

Módulos y Vite

import/export, desarrollo frente a build, variables de entorno, public/ e import dinámico

Cada fichero, su ámbito; lo que compartes, lo exportas

api.js

```js
export const BASE = "/datos";

export async function getJSON(url) { … }
export async function postJSON(url, d) { … }

export default class Api { … }
```

main.js

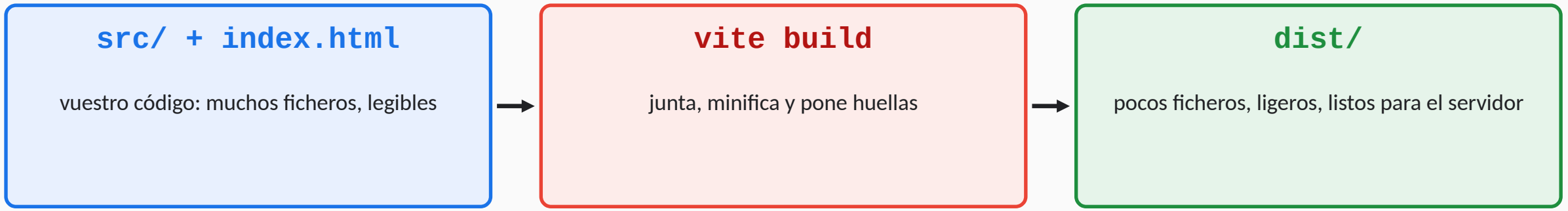
```js
import Api, { getJSON, BASE } from "./api.js";
import { getJSON as leer } from "./api.js";
import * as api from "./api.js";

// el navegador necesita la ruta COMPLETA (./ y .js);
// Vite os la completa si la olvidáis
```

Característica	Qué implica
Ámbito propio	Nada sale del fichero si no lo exportas; nada es global
Modo estricto siempre	this suelto = undefined; asignar a algo congelado lanza (UD4)
Se ejecutan una sola vez	Importar el mismo módulo en 5 sitios = una sola instancia (útil para el estado)
Diferidos	Esperan al árbol (UD5); admiten await en el nivel superior

NOTA — Exportaciones con nombre para casi todo; export default solo si el fichero tiene una cosa principal (una clase).

npm run dev para trabajar; npm run build para publicar



Orden (en Docker)	Qué hace	Cuándo
`docker compose up  (npm run dev)`	Servidor con recarga instantánea (HMR); transforma al vuelo	Mientras programáis
`… run --rm web npm run build`	Crea dist/: minificado, con nombres tipo index-4f2a.js	Antes de publicar
`… run --rm web npm run preview`	Sirve dist/ para comprobarlo como en producción	Probar el build

CLAVE — dist/ no se edita a mano ni se sube a Git (.gitignore): se vuelve a generar. Lo que se publica en un servidor estático es dist/.

.env e import.meta.env

.env.development

```
VITE_API_URL=http://localhost:3000/api
VITE_TITULO=Tablero (desarrollo)
SECRETO=no-se-expone
```

.env.production

```
VITE_API_URL=https://api.miweb.es
```

api.js

```
const BASE = import.meta.env.VITE_API_URL;

import.meta.env.MODE;   // "development" o "production"
import.meta.env.DEV;    // true en npm run dev
import.meta.env.SECRETO; // undefined: no empieza por VITE_

export const getTareas = () => getJSON(`${BASE}/tareas`);
```

¡No son secretas! Vite copia los valores VITE_ DENTRO del JavaScript de dist/: cualquiera los lee con «Ver código». Nunca claves de API privadas ni contraseñas en el frontend.

NOTA — Solo las variables con prefijo VITE_ llegan al código del navegador; las demás se quedan para la configuración de Vite.

Datos JSON servidos por el propio Vite

estructura

```
mi-proyecto/
├── index.html
├── public/
│   └── datos/
│       └── peliculas.json
└── src/
    ├── api.js
    └── main.js
```

api.js

```js
// public/datos/peliculas.json → /datos/peliculas.json
export async function getPeliculas({ lento = false } = {}) {
  if (lento) await esperar(1500);     // simular red lenta
  return getJSON("/datos/peliculas.json");
}
```

	public/	src/
Se copia a dist/	tal cual, con su nombre	procesado y con huella
Se importa con	una URL: fetch("/datos/...")	import
Para	datos JSON, favicon, robots.txt	código, CSS, imágenes que importas

CLAVE — Con un JSON en public/ practicáis fetch, estados y errores sin montar un servidor: el día que exista la API de verdad, solo cambia la URL.

Cargar código solo cuando hace falta

main.js

```
botonEstadisticas.addEventListener("click", async () => {
  // el módulo se descarga la PRIMERA vez que se pulsa
  const { pintarGrafica } = await import("./graficas.js");
  pintarGrafica(tablero.resumen());
});
```

Qué hace Vite con esto

- import() devuelve una PROMESA con el módulo
- En el build, graficas.js va en un fichero aparte (code splitting)
- La primera carga de la página es más ligera
- Ideal para lo pesado que no todos usan: gráficas, editores, mapas

Todo encaja: import() es una promesa (bloque 2), se usa con await (bloque 3), descarga un fichero por la red (bloque 4) y Vite lo prepara al construir (bloque 6).

NOTA — El import normal, el de arriba del fichero, es estático: se resuelve antes de ejecutar nada. import() se puede poner donde quieras, incluso en un if.



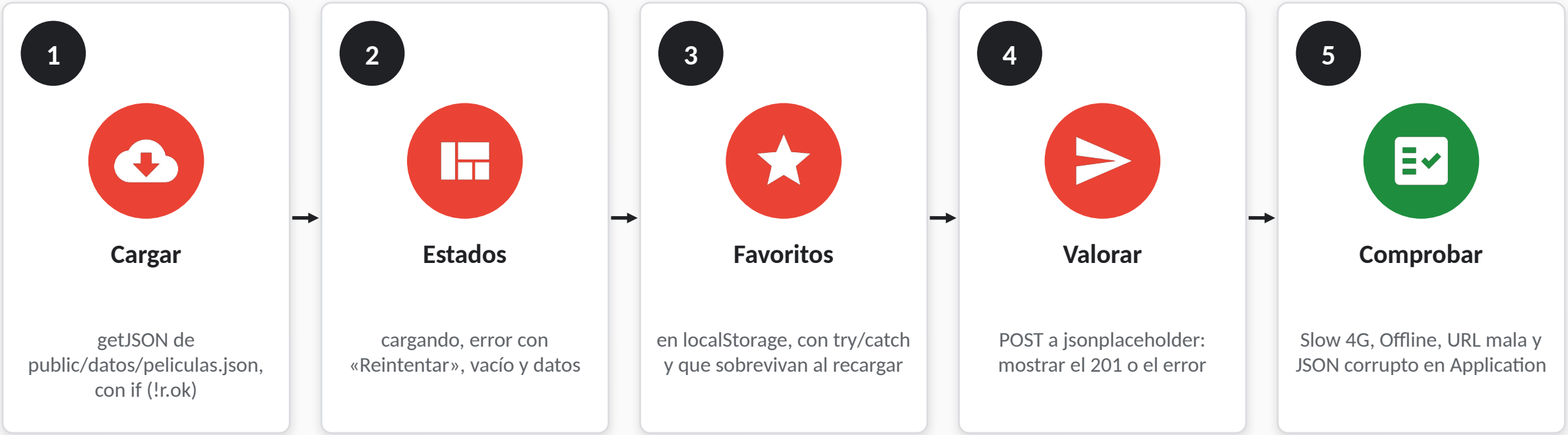
BLOQUE 7

Manos a la obra

Laboratorio, DevTools, caso real, errores comunes, control y mini-reto

LABORATORIO

Cartelera de cine con fetch, estados y favoritos guardados



Proyecto Vite en Docker. src/api.js (getJSON, postJSON), src/almacen.js (favoritos) y src/main.js con una función async iniciar(). Pintar con lo de la UD5: template y delegación.

ÉXITO — Criterio de éxito: se ve «Cargando…» con Slow 4G, un error legible con Offline o una URL mala, los favoritos siguen tras recargar y un JSON corrupto en localStorage no rompe la página.

Quién hace qué

`public/datos/peliculas.json`	8 películas: id, titulo, año, genero, duracion
`src/api.js`	getJSON(url, opciones) · postJSON(url, datos) · if (!r.ok) throw
`src/almacen.js`	cargarFavoritos() → Set · guardarFavoritos(set) · try/catch
`src/vista.js`	pintarEstado({ cargando, error, peliculas, favoritos })
`src/main.js`	estado · async cargar() · delegación (★ y Valorar) · Reintentar

Si acabáis: un buscador con debounce de 300 ms y AbortController.

main.js

```
const estado = { cargando: false,
  error: "", peliculas: [],
  favoritos: cargarFavoritos() };

async function cargar() {
  estado.cargando = true;
  estado.error = "";
  pintar();
  try {
    estado.peliculas =
      await getJSON("/datos/peliculas.json");
  } catch (e) {
    estado.error = e.message;
  } finally {
    estado.cargando = false;
    pintar();
  }
}
```

NOTA — El laboratorio es el ensayo del calentamiento C de la Práctica 2 (vie 20/11): guardar el tablero e importar tareas con fetch.

DEVTOOLS

Network y Application: la red y el almacenamiento, a la vista

`Network → Fetch/XHR`	Cada petición: método, código, tamaño y tiempo
`Clic en una petición`	Headers (lo enviado), Preview / Response (lo recibido)
`Throttling: Slow 4G`	Simula una red lenta: ¿se ve el «Cargando…»?
`Throttling: Offline`	Sin red: ¿sale el error con «Reintentar»?
`Application → Local Storage`	Ver, editar y borrar claves (prueba un JSON corrupto)
`Console`	Uncaught (in promise) = una promesa rechazada sin catch

```
Name            Status  Time
peliculas.json   200    12 ms
posts            201   310 ms
peliculs.json    404     4 ms

> localStorage
  Storage {
    "cartelera:favoritos":
      "[3,5]",
    length: 1 }
```

NOTA — Marca «Disable cache» en Network mientras programáis, o veréis respuestas antiguas del JSON de public/.

CASO REAL

Dónde os vais a encontrar todo esto

APIs REST con JSON

Casi cualquier frontend habla con su backend así: fetch + JSON + códigos HTTP.

TanStack Query, SWR

Librerías que gestionan cargando/error/caché/reintentos de las peticiones. Resuelven lo del bloque 4 a escala.

PWA y modo offline

Service Workers y la Cache API permiten que una web funcione sin red (Google Docs, Twitter Lite).

IndexedDB

Apps que guardan mucho en el navegador: editores, correo, juegos web.

Vite en la industria

Es el build de las plantillas oficiales de Vue y de buena parte de los proyectos React nuevos.

Sesiones seguras

Los tokens de sesión, en cookies HttpOnly; localStorage para preferencias, no para secretos.

DATO — Probadlo: abrid Network en cualquier web que uséis y filtrad por Fetch/XHR. Veréis su API por dentro.

ERRORES COMUNES (1/2)

Promesas y async/await

Olvidar await

Se obtiene la promesa, no el valor

```
const d = getJSON(url); d.length
```

```
const d = await getJSON(url);
```

then sin return

El siguiente then recibe undefined

```
.then((r) => { r.json(); })
```

```
.then((r) => r.json())
```

await dentro de forEach

No espera; errores sin capturar

```
lista.forEach(async (t) => await guardar(t))
```

```
for (const t of lista) await guardar(t);
```

Sin catch

Uncaught (in promise) y la interfaz colgada

```
cargar();
```

```
cargar().catch(mostrarError);
```

ERRORES COMUNES (2/2)

fetch y almacenamiento

Confiar en que fetch falla

Un 404 no rechaza

```
const d = await (await fetch(u)).json();
```

```
if (!r.ok) throw new Error(r.status);
```

body con un objeto

Se envía «[object Object]»

```
body: datos
```

```
body: JSON.stringify(datos)
```

Guardar sin stringify

Se guarda «[object Object]»

```
localStorage.setItem("t", tareas)
```

```
setItem("t", JSON.stringify(tareas))
```

parse sin protección

Un dato corrupto rompe la página

```
JSON.parse(localStorage.getItem("t"))
```

```
try { … } catch { valor por defecto }
```

CONTROL DE COMPRENSIÓN

Preguntas de control y debate

1. ¿En qué orden salen: console.log(1); setTimeout(() => log(2)); Promise.resolve().then(() => log(3)); log(4)?
2. ¿Por qué un await no congela la página, pero un while de 3 segundos sí?
3. fetch a una URL que no existe responde 404. ¿Se ejecuta el catch? ¿Qué hay que añadir?
4. Dos peticiones independientes de 1 s y 0,5 s: ¿cuánto tarda con dos await seguidos? ¿Y con Promise.all?
5. Guardas una instancia de Tarea con JSON.stringify y la recuperas con JSON.parse. ¿Qué pierdes y cómo lo arreglas?

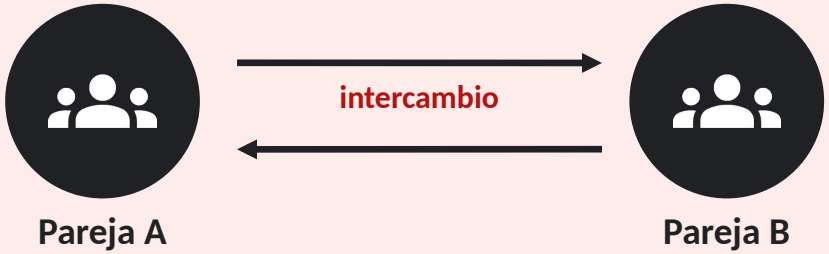
MINI-RETO PRÁCTICO

Reto express: 20 minutos, en parejas

1	**Pomodoro**	25 min: Empezar / Pausar / Reiniciar
2	**setInterval**	pinta mm:ss cada segundo
3	**Que no mienta**	calcula con Date.now(), no contando ticks
4	**Persistir**	guarda el instante de fin en localStorage
5	**Recargar**	al volver, sigue donde iba (o «¡Tiempo!»)

Revisión cruzada

A los 15 min, intercambiad el código y comprobad:



- ¿Sigue bien tras recargar a mitad?
- ¿Un solo setInterval, y se para con clearInterval?
- ¿No se adelanta ni se retrasa al cambiar de pestaña?

ÉXITO — Criterio de éxito: recargas a mitad y el contador sigue por el mismo segundo; al terminar sale «¡Tiempo!» y el intervalo se para.

GLOSARIO (1/2)

Asincronía

`síncrono`	Cada instrucción espera a que termine la anterior
`asíncrono`	Se pide algo y el resultado llega después
`pila de llamadas`	Funciones en ejecución, una encima de otra
`event loop`	Bucle que pasa callbacks de las colas a la pila
`Web APIs`	Lo que hace el navegador por ti: red, timers, DOM
`microtarea`	then/await: va antes que la siguiente tarea
`callback`	Función que se pasa para que otro la llame
`promesa`	Objeto con el resultado futuro de algo
`pending / settled`	Pendiente / ya cumplida o rechazada
`async`	Función que siempre devuelve una promesa
`await`	Pausa la función hasta resolver la promesa
`Promise.all`	Espera a todas; falla si falla una

GLOSARIO (2/2)

Red, almacenamiento y Vite

`HTTP`	Protocolo de petición y respuesta de la web
`código de estado`	2xx bien, 4xx petición, 5xx servidor
`JSON`	Formato de texto para intercambiar datos
`fetch`	API del navegador para hacer peticiones HTTP
`response.ok`	true si el código está entre 200 y 299
`AbortController`	Permite cancelar una petición
`CORS`	Permiso del servidor para leer desde otro origen
`origen`	Protocolo + dominio + puerto
`localStorage`	Almacén clave→string que dura
`toJSON`	Método que decide cómo se serializa un objeto
`build`	Preparar el código para producción (dist/)
`import.meta.env`	Variables de entorno (VITE_) en el código

ÚLTIMA REFLEXIÓN

Diseña para cuando las cosas van mal



La red falla

Lenta, caída, 404, 500. Cuatro estados en la interfaz y un error que se entienda.



El navegador no es una caja fuerte

localStorage y las variables VITE_ los puede leer cualquiera. Secretos, en el servidor.



Asíncrono no es complicado

Es pedir, seguir y recoger. async/await lo hace leerse como siempre.



Pregunta para casa: abre una app que uses a diario con Network en Offline. ¿Qué te enseña? ¿Te dice qué ha pasado y te deja reintentar?

CHECKPOINT

Resumen de puntos clave

- Un hilo: lo lento se pide y llega después (event loop)
- Síncrono → microtareas (then/await) → tareas (timers, eventos)
- async/await con try/catch; Promise.all si son independientes
- fetch no falla con un 404: comprueba response.ok
- localStorage: strings, JSON, try/catch, nunca secretos
- Vite: dev para trabajar, build → dist/ para publicar

SIGUIENTE · UD7

React: componentes y JSX

- El render() que hacíais a mano, automático
- Componentes en vez de crearItem
- Mismo DOM, mismos eventos, mismo fetch

Primer trimestre: JavaScript sin frameworks, de la sintaxis a una app completa que habla con la red.